

Corridor Baseline Learning

Lanjutan dari Historical Shipment Store: setiap laporan hasil aktual pengiriman kini memperbarui rata-rata delay & tingkat kerusakan nyata per koridor, dan prediksi berikutnya secara bertahap mempercayai data nyata itu dibanding formula sintetis.

Branch: fix/hackathon-readiness

Service: services/ai/route-planner

Tanggal dokumentasi: 22 Agustus 2026

Melanjutkan: docs/historical-shipment-store.pdf (fitur penyimpanan shipment record)

1. Latar Belakang

Pembaruan sebelumnya (Historical Shipment Store) membangun sisi *penulisan*: setiap `/predict-route` tersimpan sebagai shipment record, dan `PATCH /shipments/{id}/outcome` memungkinkan pelaporan hasil aktual (delay & status kerusakan sungguhan). Namun sisi *pembacaan* masih kosong — fitur `historical_delay_avg_hours` dan `historical_damage_rate` yang dipakai model risiko tetap 100% dihitung dari formula deterministik di `app/services/historical_baseline.py`, tanpa pernah melirik data nyata yang sudah mulai terkumpul.

TUJUAN PEMBARUAN INI

Menutup celah itu: setiap kali ada laporan outcome baru, sistem memperbarui agregat data nyata per koridor rute, dan prediksi berikutnya pada koridor yang sama secara bertahap "percaya" pada data nyata tersebut — bukan lompatan tiba-tiba, tapi ramp kepercayaan yang halus seiring jumlah laporan bertambah.

BATASAN YANG DISENGAJA

Ini scope-nya murni memperbarui formula baseline delay/kerusakan, **bukan** melatih ulang model XGBoost. Retraining model butuh pipeline data yang jauh lebih berat (export, validasi kualitas minimum, versioning model) dan datanya sendiri baru mulai terkumpul dari nol hari ini — belum ada cukup data untuk melatih ulang secara bermakna. Keputusan scope ini sudah dikonfirmasi bersama pengguna.

2. Keputusan Desain

2.1 Kunci koridor: (moda transportasi, bucket jarak 100km) — tanpa exposure_tier

`estimate_historical_baseline()` selama ini mencampur tingkat eksposur gelombang secara berbobot (`exposure_tier_weights()`), bukan satu tier tunggal per pengiriman — jadi satu shipment record tidak punya "tier tunggal" untuk dijadikan kunci. Memecah agregat berdasarkan tier juga akan memecah sampel data nyata yang memang masih sedikit menjadi tiga bagian sebelum ada data sama sekali. Kunci yang lebih kasar — hanya (moda, bucket jarak) — mempercepat titik "cukup data untuk dipercaya", yang penting karena tabelnya dimulai kosong hari ini.

`distance_bucket_km = round(distance_km / 100) * 100` — bin 100 km, selaras dengan sensitivitas formula sendiri ($0.01 \times \text{distance_km}$), sehingga variasi formula di dalam satu bucket kecil.

2.2 Tabel baru `corridor_baseline_stats`, diperbarui hanya saat outcome dilaporkan

"Data baru masuk ke historical store" punya dua momen: prediksi tercatat (`predicted_only`, belum ada kebenaran lapangan — tidak ada yang bisa dipelajari) dan outcome dilaporkan (`outcome_reported` — titik data nyata pertama). Hanya momen kedua yang bisa "dipelajari", jadi update agregat dipasang di `shipment_service.report_outcome()`, bukan di `record_prediction()`.

Disimpan sebagai jumlah kumulatif (`sum`), bukan rata-rata langsung — menghindari drift floating-point dan membuat update jadi satu operasi atomik `INSERT ... ON CONFLICT DO UPDATE`, pola yang sama seperti `upsert bmkp_cache` yang sudah ada.

2.3 Blend bertahap, bukan cutover mendadak

Beralih total ke data nyata begitu ambang tertentu tercapai akan menghasilkan lonjakan nilai yang kasar (step function) hanya dari beberapa sampel. Sebagai gantinya, tingkat kepercayaan naik linear:

Jumlah laporan (sample_count)	Bobot data nyata	Keterangan
0 – 2	0%	Murni formula sintetis (belum cukup data)
3	0%	Ambang minimum tercapai, mulai bisa dipakai laporan berikutnya
11 (contoh tengah)	~47%	Campuran formula & data nyata
≥ 20	100%	Sepenuhnya memakai rata-rata data nyata

```

MIN_SAMPLES_FOR_BLEND = 3
FULL_TRUST_SAMPLES = 20

def _blend_weight(sample_count):
    if sample_count < MIN_SAMPLES_FOR_BLEND:
        return 0.0
    span = FULL_TRUST_SAMPLES - MIN_SAMPLES_FOR_BLEND
    return min(1.0, (sample_count - MIN_SAMPLES_FOR_BLEND) / span)

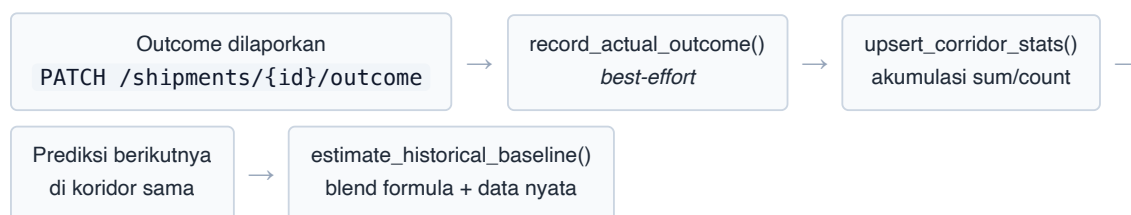
final_value = (1 - weight) * formula_value + weight * real_average

```

2.4 Best-effort, konsisten dengan seluruh akses DB lain di service ini

`estimate_historical_baseline()` dipanggil sekali per kandidat rute pada **setiap** `/predict-route` dan `/simulate-scenario` (lewat `enrichment_service.py`), dan sebelumnya adalah fungsi murni tanpa I/O apa pun. Menambahkan pembacaan DB di sini berarti setiap prediksi kini melakukan round-trip ke Postgres — dibungkus `try/except` dan otomatis jatuh kembali ke formula murni bila DB bermasalah, sama seperti prinsip yang sudah ditetapkan untuk `shipment_records` sebelumnya. **Pipeline training (`train_model.py`, `synthetic_corridors.py`) sama sekali tidak tersentuh** — keduanya hanya mengimpor fungsi formula murni (`expected_base_delay_hours / expected_base_damage_rate`), bukan `estimate_historical_baseline()`, sehingga tetap 100% offline seperti sebelumnya.

3. Alur Data



4. Skema Tabel `corridor_baseline_stats`

Kolom	Tipe	Keterangan
<code>transport_mode</code>	String (PK)	<code>darat</code> / <code>laut</code> / <code>kombinasi</code>
<code>distance_bucket_km</code>	Integer (PK)	Jarak dibulatkan ke bin 100 km terdekat
<code>sample_count</code>	Integer	Jumlah laporan outcome yang masuk ke koridor ini
<code>delay_hours_sum</code>	Float	Jumlah kumulatif <code>actual_delay_hours</code> dari semua laporan
<code>damage_occurred_count</code>	Integer	Jumlah laporan dengan <code>actual_damage_occurred = true</code>
<code>updated_at</code>	DateTime(tz)	Waktu update terakhir

Rata-rata dihitung saat baca: `real_delay = delay_hours_sum / sample_count`, `real_damage_rate = damage_occurred_count / sample_count`.

5. Fungsi Baru

Fungsi	File	Peran
<code>upsert_corridor_stats()</code>	<code>app/core/db.py</code>	Upsert atomik: tambah 1 sampel + akumulasi sum delay/kerusakan untuk kombinasi (moda, bucket jarak)
<code>get_corridor_stats()</code>	<code>app/core/db.py</code>	Ambil agregat saat ini untuk satu koridor, atau <code>None</code> jika belum ada data
<code>_distance_bucket_km()</code>	<code>app/services/historical_baseline.py</code>	Bulatkan jarak ke bin 100 km
<code>_blend_weight()</code>	<code>app/services/historical_baseline.py</code>	Hitung bobot kepercayaan data nyata dari jumlah sampel
<code>record_actual_outcome()</code>	<code>app/services/historical_baseline.py</code>	Dipanggil dari <code>report_outcome()</code> — upsert best-effort ke <code>corridor_baseline_stats</code>
<code>estimate_historical_baseline()</code>	<code>app/services/historical_baseline.py</code>	(dimodifikasi) setelah hitung formula, cek agregat nyata dan blend jika sampel cukup

6. Pengujian & Hasil Verifikasi

6.1 Strategi tes

- Unit test murni (tanpa DB):** `test_blend_weight_ramps_from_zero_to_full_trust` memverifikasi kurva bobot 0% → 100% pada fungsi `_blend_weight()` langsung, tanpa DB sama sekali.
- Best-effort fallback (tanpa DB):** `test_predict_route_survives_unreachable_corridor_stats` memaksa `get_corridor_stats` melempar error koneksi, memastikan `/predict-route` tetap 200 OK dan `historical_delay_avg_hours` tetap terisi dari formula.

3. Uji "belajar" end-to-end (Postgres live): `test_corridor_baseline_learns_from_reported_outcomes` — melapor 5 outcome dengan `actual_delay_hours = 999` (nilai jauh dari formula) pada satu koridor tetap (Bandung → Jakarta, darat), lalu memanggil `/predict-route` lagi pada koridor yang sama dan memverifikasi `historical_delay_avg_hours` bergeser signifikan ($> \text{formula} + 50$ jam) menuju nilai yang dilaporkan.

6.2 Hasil aktual dijalankan

Suite	Kondisi	Hasil
Seluruh test suite <code>services/ai/route-planner/tests/</code> (39 test)	Tanpa Postgres live	36 passed, 3 skipped (tes yang memang butuh DB nyata)
<code>tests/golden_demo/test_golden_demo.py</code>	Tanpa Postgres live	1 passed — tidak ada perubahan diperlukan
<code>test_shipment_store_e2e.py</code> (9 test, termasuk 3 test baru fitur ini)	Dengan Postgres live	9 passed — termasuk uji "belajar" end-to-end
<code>training/synthetic_corridors.py</code> (<code>build_corridors()</code> , 200 koridor)	Postgres dimatikan sepenuhnya	Berhasil dibangun tanpa error — membuktikan pipeline training tetap 100% offline

KESIMPULAN VERIFIKASI

Fitur ini murni menambah kemampuan baru tanpa mengubah perilaku jalur yang sudah ada saat database tidak tersedia, dan tidak menyentuh sama sekali pipeline training model.

7. File yang Diubah & Ditambahkan

[diubah] `services/ai/route-planner/app/core/db.py`

[diubah] `services/ai/route-planner/app/services/historical_baseline.py`

[diubah] `services/ai/route-planner/app/services/shipment_service.py`

[diubah] `services/ai/route-planner/tests/test_shipment_store_e2e.py`

8. Contoh Skenario Demo

- POST `/predict-route` untuk rute darat tetap (mis. Bandung → Jakarta) — catat `historical_delay_avg_hours` awal (murni formula, karena koridor belum punya data).
- PATCH `/shipments/{id}/outcome` 3–5 kali untuk pengiriman di koridor yang sama, dengan `actual_delay_hours` yang konsisten menyimpang dari estimasi formula.
- POST `/predict-route` lagi pada koridor yang sama — `historical_delay_avg_hours` kini bergeser mendekati nilai yang dilaporkan, sebanding dengan jumlah laporan yang sudah masuk.

Ini yang membuat fitur ini terasa "belajar" secara nyata dalam demo: semakin banyak laporan hasil aktual masuk ke satu koridor, semakin besar pengaruhnya terhadap prediksi berikutnya di koridor itu — sampai penuh dipercaya pada 20 laporan.